

Campus Lost and Found Management System

Project Report

Submitted by	Srijal Kumar
Registration Number	26BCE10659
Course and Branch	B.TECH (Computer Science and Engineering)
Semester	1st Semester
Subject	Introduction to Problem Solving and Programming
Course Code	CSE1021
Faculty name	Mohd Rafi Lone
Institution Name	VIT Bhopal University

1. Introduction

Many students in our college deal with losing items in the campus — ID cards, water bottles, notebooks, electronics, and more. Most of the time students just hope that someone will return their item or they rely on notice boards which barely works. There is no proper way to report , search and keep track so that students can get their stuff back .

This is why I built this Campus Lost and Found Management System, a command-line Python system that gives students a simple, structured way to report lost items, report found items, search for an item, mark an item as returned, and delete incorrect entries. I build this project as part of the first-semester CSE1021 - Introduction to Problem Solving and Programming - Flipped Course, applying core concepts such as functions, lists, loops, conditionals, and modular file organization.

2. Problem Statement

Right now, there's no easy, all-in-one system on campus for students to log and track lost or found items. We mostly rely on clunky notice boards, random WhatsApp groups, or word of mouth, but these ways are pretty disorganized. It's hard to search, and items aren't tracked properly. My project tackles this by offering a lightweight, menu-based console system. You can report, search, and update lost or found items, all in one place, for as long as the session runs.

3. Objectives

- Make it dead simple for students to report lost items and where they lost them.
- Allow anyone (students or staff) to report things they've found.
- Enable users to search for an item by name to quickly check if it's been reported.
- Mark items as 'Returned' once the owner gets them back.
- Let users delete mistakes or duplicate entries.
- Put Python fundamentals-like functions, lists, loops, and modularity-into practice in a real-world-ish project.

4. Functional Requirements

The system provides the following functional modules:

Module	File	Description
Report Lost Item	lost.py	Takes item's name and place, adds it to the lost items list with status 'Lost'.
Report Found Item	found.py	Takes item name and place, adds it to the found items list with status 'Found'.
View Items	lost.py / found.py	Shows all current lost or found items with their status.
Search Item	search.py	Searches both lost and found lists by item name and displays matching results.
Mark as Returned	returned.py	Updates status to “Returned” for an item, that matches the name and place.
Delete Item	delete.py	Removes the wrong or duplicate entry from the system.

5. Non-Functional Requirements

Requirement	How it is addressed
Usability	A numbered menu loops back after every action and user can use other menu options if they want.
Maintainability	Each feature is made and stored into its own file (module), so that code is easy to read, and expand.
Reliability	If a user enters something invalid, the program handles it with a “Invalid Choice” message and doesn't crash.
Performance	Everything runs in a memory using Python's lists. For a single session, the actions are nearly instant.

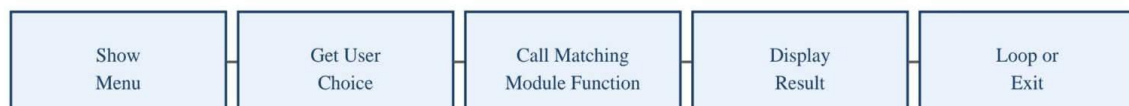
Note: Please note that when you close the program, all data saved in the program will be gone. So it is recommended to saving it to a file. Data persistence has left for future enhancement on this project.

6. System Architecture

The system follows a basic modular architecture. `main.py` acts as the controller: it displays the menu, collects the user's choice, and calls the right function from the appropriate module. Each module handles one responsibility, and all modules share two lists : (`lost_items` and `found_items`) .

7. Process Flow / Workflow

The system follows this repeating workflow for every user action:



8. Use Case Description

Who uses it ? Student or Staff Members - anyone can report, view, edit, or delete items, right from the menu:

- Report Lost Item
- Report Found Item
- View Lost Items
- View Found Items
- Search Item
- Mark Item as Returned
- Delete Lost Item
- Delete Found Item

Note : For now, it's a single - user console app. No logins, no user roles - everyone has access to all features. Splitting roles is on my list for future versions.

9. Module Diagram

Module	Function
main.py	Program entry point, displays menu, handles user input
lost.py	report_lost(lost_items) view_lost(lost_items)
found.py	report_found(found_items) view_found(found_items)
search.py	search_item(lost_items, found_items)
returned.py	marked_returned(lost_items, found_items)
delete.py	delete_lost(lost_items) delete_found(found_items)

10. Sequence

Example - Reporting a Lost Item

Here's what happens when you report a lost item:

- User selects option 1 from the menu in main.py.
- main.py calls report_lost(lost_items) from lost.py.
- report_lost() asks the user for the item name and place.
- It builds a list [name, place, "Lost"] and add it to lost_items.
- A confirmation message is printed.
- Backs to main menu

11. Implementation Details

The project is split into 6 Python files. main.py keeps the two shared lists (lost_items, found_items) and the uses a while True loop to show the menu and branch on choice. All the other modules are imported. Reporting functions(report_lost, found_lost) take a list, build a 3-element list per item, and add it. search_item checks both lost and found, matching case-insensitively with .lower(). The marked_returned and delete functions search by name and place (so if you lose two similar items in different spots, you don't delete or return the wrong one).

12. Sample Program Output

Below is an actual run of the program covering some of the core features of the program and we take an example for a *lost Item* in the campus:

```
-----  
CAMPUS LOST AND FOUND MANAGEMENT  
-----  
  
1.Report Lost Item  
2. Report Found Item  
3. View Lost Items  
4. View Found Items  
5. Search Items  
6. Mark Item As Returned  
7. Delete lost Item  
8. Delete Found Item  
9. Exit  
  
Enter your Choice : 1
```

Fig no. 1 Main menu of the Program

```
-----  
CAMPUS LOST AND FOUND MANAGEMENT  
-----  
  
1.Report Lost Item  
2. Report Found Item  
3. View Lost Items  
4. View Found Items  
5. Search Items  
6. Mark Item As Returned  
7. Delete lost Item  
8. Delete Found Item  
9. Exit  
  
Enter your Choice : 1  
  
-----Report Lost Item-----  
Enter Item Name : phone  
Where did you lose it? : AB1  
Lost Item Added Successfully.
```

Fig no. 2 Report a Lost Item

```
-----  
CAMPUS LOST AND FOUND MANAGEMENT  
-----  
  
1.Report Lost Item  
2. Report Found Item  
3. View Lost Items  
4. View Found Items  
5. Search Items  
6. Mark Item As Returned  
7. Delete lost Item  
8. Delete Found Item  
9. Exit  
  
Enter your Choice : 3  
  
-----Lost Items-----  
Item : phone  
Place : AB1  
Status : Lost
```

Fig no.3 View lost Item

```
-----  
CAMPUS LOST AND FOUND MANAGEMENT  
-----  
  
1.Report Lost Item  
2. Report Found Item  
3. View Lost Items  
4. View Found Items  
5. Search Items  
6. Mark Item As Returned  
7. Delete lost Item  
8. Delete Found Item  
9. Exit  
  
Enter your Choice : 6  
  
----- Mark Item as Returned-----  
Enter Item Name : phone  
Enter place : ab1  
Item Marked as Returned.
```

Fig no. 4 Marked Item as Returned

```
-----  
CAMPUS LOST AND FOUND MANAGEMENT  
-----  
  
1.Report Lost Item  
2. Report Found Item  
3. View Lost Items  
4. View Found Items  
5. Search Items  
6. Mark Item As Returned  
7. Delete lost Item  
8. Delete Found Item  
9. Exit  
  
Enter your Choice : 7  
  
-----Delete Lost Items-----  
Enter Item name to delete : phone  
Enter place : ab1  
Item Deleted Successfully.
```

Fig no. 5 Delete a Lost Item

```
-----  
CAMPUS LOST AND FOUND MANAGEMENT  
-----  
  
1.Report Lost Item  
2. Report Found Item  
3. View Lost Items  
4. View Found Items  
5. Search Items  
6. Mark Item As Returned  
7. Delete lost Item  
8. Delete Found Item  
9. Exit  
  
Enter your Choice : 9  
Program Ended . Thank You!!!
```

Fig no. 6 End of the Program

13. Testing Approach

Testing was done manually by running the program and trying each menu option with normal and edge-case inputs.

Test Case	Input	Expected Result	Actual Result
Report a lost item	Name + Place	"Item added" , confirmation shown	PASS
Report a found item	Name + Place	"Item added" , confirmation shown	PASS
View items with empty list	No items reported yet	"No lost/found items" message	PASS
Search for existing item	Correct item name	Item details displayed	PASS
Search for non existent item	Random name	"Item not found" message	PASS
Mark item as returned	Correct Name + Place	Status changes to "Returned"	PASS
Delete an item	Correct Name+ Place	Item removed from list	PASS
Delete from empty list	No items exist	"No items to delete" message	PASS
Invalid menu choice	e.g. 156	"Invalid Choice Entered" message	PASS

14. Challenges Faced

- Figuring out Python lists for storing and updating lost/found items took some trial and error, especially when I had to make sure information showed up right every time.
- Linking up menu choices and functions without the program getting stuck or crashing wasn't easy. Making sure every function returned properly to the main loop was key.
- Search and Return features turned out trickier than expected - I had to really pay attention to loops and matching exact records so nothing went wrong if there were multiple similar items.

15. Learnings & Key Takeaways

- I Learned how to split a program across multiple files (modules) and import functions between them.
- I got comfortable with passing lists (mutable objects) between functions and changing them in place.
- Designing a simple, menu-driven CLI system from zero really helped.
- Planning out what the app should do - both functionality and behind the scenes - definitely paid off when coding.

16. Future Enhancements

- Add file saving, maybe (JSON or CSV) so data is saved permanently after quitting.
- Assign a unique ID to each item so we don't have to rely only on name and place.
- Suggest possible matches automatically when a found item lines up with a lost report.
- Switch from procedural to object - oriented programming..
- Build a GUI or basic web interface instead of just CLI.
- Connect to a database(like SQLite) to make data more reliable and scalable.

17. References

1. Vityarthi Course Modules - For learning Python Programming concepts and core topics that are used in the project.
2. Introduction to Problem Solving and Programming (CSE1021) Class lectures and Notes - Used for understanding fundamental programming concepts, algorithms, functions, lists, loops and conditional statements.